FinVerity on PureChain: Experimental Setup and Results

Blockchain-anchored, continuously verifiable financial audit evidence

Generated from the complete measurement campaign (777 trials)

Abstract

This report documents the experimental methodology and the measured results for FinVerity, an audit-evidence architecture that commits SHA-256 digests of canonicalised financial evidence packages to the PureChain PoA^2 blockchain while the evidence itself remains off-chain. Four transaction-recording infrastructures are compared under identical workloads derived from public synthetic financial transaction datasets: a centralised database (PostgreSQL), a conventional permissioned blockchain (Hyperledger Besu QBFT), a conventional public-chain configuration (Ethereum proof-of-stake with mainnet timing) and PureChain, measured both on the live network and on a PoA^2 emulation calibrated to it. The evaluation covers performance (latency, finality, throughput), efficiency (energy, cost, resources) and auditability (verification, tamper detection, insider resistance), and every core experiment is replicated on three independent transaction datasets (MoMTSim, SAML-D, IBM AML). All tables and figures are generated directly from the raw measurements; Table 6 states which experiments are complete.

1 Experimental Setup

1.1 Research design

The experiments address the question whether a lightweight, zero-gas, PoA^2-based permissioned blockchain can provide cryptographically verifiable financial audit evidence while reducing transaction latency, energy consumption and operational cost. Equivalent workloads are executed on every infrastructure and three groups of metrics are collected: *performance* (latency, finality, throughput), *efficiency* (energy, cost, CPU, memory, network, storage) and *auditability* (verification success, evidence–transaction binding, tamper detection, insider resistance).

Table 1: Evaluation framework: metric groups and their operationalisation.

Group	Metric	Operationalisation
Performance	Throughput (TPS)	committed records / (last inclusion − first arrival)
	Transaction latency	arrival → first observation of the canonical block containing the record
	Finality latency	arrival → finality under the rule of Table 2
	Confirmation time	arrival → observation of block $b+k$, $k \in \{1,2,3,6\}$
Efficiency	Energy per transaction (J/tx)	attributed node energy and measured dynamic CPU energy per record
	CPU utilisation	CPU-seconds of client and infrastructure containers per second of processing
	Memory utilisation	peak resident memory of infrastructure containers
	Network I/O	request and response bytes per record
	Storage overhead	on-chain bytes per record; off-chain evidence bytes per record
	Operational cost per transaction	$C_{compute} + C_{storage} + C_{network} + C_{fee}$
Auditability	Evidence verification time	record lookup + evidence retrieval + re-hash, per record
	Verification success rate	share of committed records with $H_i' = H_i$
	Tamper detection rate	TDR over eleven modification classes; FPR over benign transformations
	Evidence–transaction binding	identifier, commitment and approver all match
	Insider resistance	attacks by a privileged operator prevented or detected

1.2 Infrastructures under test

Table 2: Infrastructures compared. All blockchain configurations run the same Solidity contract (`FinVerityRegistry`, solc 0.8.19, EVM target *paris*).

Configuration	Consensus / engine	Nodes	Finality rule	Fees
PureChain (live)	PoA^2 (Geth 1.13.15 fork, Clique-style seals)	4 validators (remote, public RPC)	6 confirmations[a]	gas price 0
PoA^2 emulation	Clique PoA, Geth 1.13.15, 1 s period	4 signers (Docker, 1 CPU each)	6 confirmations[a]	fee-free (1 wei nominal)[b]
Besu QBFT	QBFT BFT, Besu 24.12.2, 1 s period	4 validators (Docker, 1 CPU each)	immediate (inclusion)	gas price 0
Ethereum PoS	Casper FFG + LMD-GHOST; Geth 1.13.15 + Prysm 5.0.4	1 node, 64 validators	finalized checkpoint	EIP-1559[c]
PostgreSQL 16	single operator, `fsync=on`, synchronous commit	1 server (Docker)	durable commit	none

[a]Forks up to depth 5 were observed on live PureChain (Section 2.1); $k = 6$ is the smallest depth exceeding every observed reorganisation. [b]Upstream Geth enforces a 1 wei floor for transactions relayed between peers; the cost is economically zero ($\approx 1.5 \times 10^{-13}$ ETH per record) and no fee is charged in the cost model. [c]Fees are expressed at a mainnet reference price of 5 gwei and 3,000 USD/ETH.

Every blockchain configuration uses a 30 M block gas limit. Besu and the PoA^2 emulation give every validator the same 1-CPU quota, so that validator-count scaling is not confounded by contention on the single test host. The Ethereum PoS configuration uses mainnet timing (12 s slots, 32 slots per epoch) and represents the *conventional public-chain configuration* named in the hypothesis. Only the backend under test runs during a trial; all other local networks are stopped.

1.3 Testbed

Client and local infrastructure run on one workstation: Intel Core i5-12400F (6 cores, 12 threads, 65 W base power), 32 GB RAM, Windows 11, Docker Desktop 29.4 (12 CPUs, 16.7 GB assigned). Software: Python 3.13, `web3.py` 6.20.4, `purechainlib` 2.1.7, PostgreSQL 16, Hyperledger Besu 24.12.2, Geth 1.13.15, Prysm 5.0.4. Live PureChain is reached over the public RPC endpoint (chain ID 900520900520, median RPC round trip 15.7 ms). The measured network parameters of PureChain (4 validators sealing round-robin, 71% of blocks sealed in turn, 30,000,000 block gas limit, gas price 0) were obtained by recovering the signer of each block from its Clique-style seal.

1.4 Financial transaction workloads

The datasets are used strictly as *financial transaction workloads*; fraud and laundering labels are discarded.

Table 3: Transaction workload datasets.

Dataset	Transactions	Period	Main types	Median amount
MoMTSim (primary; Azamuke et al., 2024)	4,225,958	8 simulated days	transfer, payment	596.69
SAML-D (Oztas et al., 2023)	9,504,852	Oct 2022–Aug 2023	card, cheque, ACH	6,113.72
IBM AML HI-Small (Altman et al., 2023)	5,078,345	Sep 2022	cheque, card, ACH	1,414.54

Using three datasets with different transaction mixes, currencies, amount distributions and account structures tests whether the results depend on the workload source. MoMTSim drives all experiments; the core workloads W1, W2 and W5, the tamper-detection experiment and the insider experiment are replicated on SAML-D and IBM AML (E6–E8). All datasets are mapped to a common schema (identifier, timestamp, type, amount as a decimal string, currency, originator, beneficiary, balances, channel). MoMTSim provides neither identifiers nor wall-clock times; both are derived deterministically from the row index and the simulated hour. Each trial draws a contiguous chronological window at a seeded random offset, which preserves temporal ordering and account interactions.

1.5 Evidence generation and cryptographic commitment

For every transaction T_i a synthetic audit-evidence package $E_i = \{M_i, D_i, A_i, L_i, C_i\}$ is generated deterministically: metadata M_i; invoice and purchase-order record D_i with supporting documents; authorisation record A_i (approval tier by amount, approver, segregation of duties); balanced double-entry journal L_i; and

context C_i. Complexity levels are L1 (metadata), L2 (+invoice), L3 (+authorisation) and L4 (complete). Supporting documents are incompressible byte streams of controlled size (0, 10 KB, 100 KB, 1 MB, 5 MB), each represented in the package by its SHA-256 digest. The commitment is

$$E_i^* = \mathrm{Canon}(E_i), \qquad H_i = \mathrm{SHA256}(E_i^*), \tag{1}$$

where Canon is an RFC 8785 profile (sorted keys, no insignificant whitespace, UTF-8, Unicode NFC, monetary values as decimal strings, floats rejected). An approver signs (EIP-191) the domain-bound digest

$$S_i = \mathrm{Sign}_{approver}\big(\mathrm{keccak}(\mathit{chainId} \parallel \mathit{registry} \parallel ID_i \parallel H_i \parallel \mathit{metaHash} \parallel \mathit{level})\big). \tag{2}$$

The contract accepts `recordEvidence` only from registered submitters with a valid approver signature, rejects duplicate identifiers and stores $\{H_i, \mathit{metaHash}, \mathit{submitter}, \mathit{approver}, t_i, \mathit{block}\}$. In batched mode (`recordBatch`) k records are committed as one Merkle root over leaves $\mathrm{keccak}(ID_i \parallel H_i)$; each record keeps an inclusion proof, verifiable off-chain or with the contract's `verifyInclusion`. The evidence is kept off-chain in a durable store (SQLite, WAL, `synchronous=FULL`).

1.6 Submission and measurement

Records arrive as an open-loop Poisson process at the offered rate. Transactions are signed in advance and submitted by 16 submitter accounts in parallel over persistent keep-alive connections. A batch closes when it holds k records or its first record has waited 1 s. A fork-aware block watcher polls the chain head every 50 ms (250 ms for PoS), records every block version with its first-observation time and walks back along parent hashes, so replaced blocks are captured. Final transaction status is taken from receipts re-read against the canonical chain. Latencies are measured per record from its arrival:

- *inclusion latency*: arrival to first observation of the canonical block containing the record;
- *k-confirmation latency*: arrival to observation of block $b + k$;
- *finality latency*: arrival to finality under the rule of Table 2;
- *throughput*: committed records divided by the time from the first arrival to the last inclusion.

1.7 Efficiency metrics

Energy is reported in two ways for every backend. The *attributed* energy $E_{tx} = \big(n_{nodes}\, P_{node}\, T_{proc} + E_{client}\big)/N$ charges every infrastructure node ($P_{node} = 25$ W, $n_{nodes} = 4$ for blockchains, 1 for the database) for the processing window T_{proc} from the first arrival to the last inclusion, so waiting for finality is not charged. The *measured dynamic* energy converts CPU time of the client and of the local containers into energy with a linear power model $E = P_{dyn} \cdot t_{CPU}/n_{cores}$ (Fan et al., 2007; $P_{dyn} = 65$ W, 12 threads). Cost is $C_{tx} = C_{compute} + C_{storage} + C_{network} + C_{fee}$, with 0.12 USD/kWh, 0.023 USD/GB-month off-chain storage, 0.08 USD/GB-month on-chain storage per replica, 0.09 USD/GB network traffic and an 84-month retention period. C_{fee} is non-zero only for the fee-bearing public-chain configuration.

1.8 Auditability metrics

The auditor retrieves H_i (or the batch root and proof), recomputes $H_i' = \mathrm{SHA256}(\mathrm{Canon}(E_i'))$ after re-hashing every stored document, and sets $\mathrm{Verify}(E_i') = 1$ iff $H_i' = H_i$. Binding success also requires the identifier and the recorded approver to match the package. The tamper detection rate is

$$\mathrm{TDR} = \frac{\text{detected modifications}}{\text{applicable modifications}} \times 100, \tag{3}$$

over eleven malicious modification classes (amount, date, invoice number, vendor, authorisation, supporting document, deleted document, ledger entry, beneficiary, single-bit change, record substitution). Two benign transformations (re-serialisation with reordered keys, Unicode NFD) give the false-positive rate. The insider experiment assumes an operator holding submitter and approver credentials who (S1) alters evidence only, (S2) alters evidence and overwrites the stored commitment, or (S3) fabricates and backdates a replacement record.

Table 5: Calibration of the PoA^2 emulation against live PureChain (identical workload: 300 MoMTSim records, 10 KB evidence, 50 records/s Poisson arrivals).

Metric	Live PureChain	PoA^2 emulation	Deviation
Records committed	300	300	+0.0%
Throughput (records/s)	36.7	40.0	+8.9%
Inclusion latency, median (s)	2.24	1.49	-33.2%
Inclusion latency, P95 (s)	3.22	1.96	-39.2%
1-confirmation latency, median (s)	3.07	2.48	-19.2%
3-confirmation latency, median (s)	4.29	4.48	+4.4%
6-confirmation latency, median (s)	7.19	7.47	+3.8%
Transactions per non-empty block, mean	42.9	42.9	+0.0%
Distinct block signers	4	4	+0.0%
Gas per record	153,317	153,434	+0.1%
Verification success (%)	100.0	100.0	

Live inclusion latency additionally contains the wide-area round trip and the public load balancer.

1.9 Experimental design

Table 4: Experiments (all use evidence level L4 unless varied).

ID	Factor	Configuration
E1	workload	W1 1k/10 KB (n=30), W2 10k/10 KB (10), W3 50k/100 KB (5), W4 100k/100 KB (3), W5 10k/1 MB (10); 50 rec/s
E2	offered load	10, 25, 50, 100, 200, 400 rec/s and unthrottled; 5 trials
E3	evidence size	none, 10 KB, 100 KB, 1 MB, 5 MB; 1,000 records; 5 trials
E4	complexity	L1–L4; 1,000 records, 100 KB; 5 trials
E5	validators	4, 7, 10 (Besu and PoA^2 emulation) at 50 and 200 rec/s; 5 trials
E6	tamper detection	1,000 records per dataset, 13 transformations per record; 3 trials per dataset
E7	dataset	W1 (n=10), W2 (5) and W5 (3) replicated on SAML-D and IBM AML
E8	insider	300 records, 100 attacked per scenario; 3 trials (MoMTSim), 2 per secondary dataset
E9	batch size	$k \in \{1, 10, 25, 50, 100\}$ at 50 and 200 rec/s; 5 trials

1.10 Statistical analysis

Each configuration is repeated in independent trials with different seeded workload windows. Trial-level results are reported as mean with a 95% t-interval; latency distributions as median, P95 and P99. Backends are compared with Welch's t-test and the Mann–Whitney U test with Holm–Bonferroni correction and Cliff's δ as effect size. Detection rates are reported with exact Clopper–Pearson intervals.

1.11 PoA^2 emulation and its calibration

The public PureChain RPC endpoint stopped producing blocks during the campaign (Section 4). To complete the controlled experiments, a PoA^2 emulation was built from the parameters measured on the live network and validated against a live run with an identical workload (Table 5). Block packing and confirmation-depth latency agree closely; inclusion is faster on the emulation because the wide-area path and the public load balancer are absent. The emulation did not reproduce the forks seen on the live network under heavy load; applying the same 6-confirmation rule to both keeps the emulated finality latency conservative. Results from the emulation are labelled as such throughout.

2 Results

2.1 Live PureChain characterisation

Block seals recovered from the live chain show 4 authorised validators sealing in round-robin, with in-turn (difficulty 2) and out-of-turn (difficulty 1) blocks, which is Clique-style proof-of-authority behaviour. Blocks are produced at about 1 s intervals while transactions are pending and the chain idles otherwise. Under sustained load, competing blocks at the same height were produced by different signers: over 15 live trials, 43 of 183 observed blocks (23.5%) were replaced by a competing block, with forks up to 5 blocks deep. Every transaction contained in a replaced block was re-included and verified. Inclusion therefore gives only probabilistic finality, and 6 confirmations are used as the finality rule for PoA^2 (live and emulated). Shortly after the heaviest load the public network stopped producing blocks (Section 4).

Table 6: Campaign completion status (completed / planned trials) at the time this report was generated. PureChain (live) trials are pending the recovery of the public RPC endpoint.

ID	Experiment	PureChain (live)	PoA^2 emulation	Besu QBFT	PostgreSQL	Ethereum PoS
E1	Controlled workloads W1–W5	0/58	58/58	58/58	58/58	7/7
E2	Offered-load sweep	0/35	35/35	35/35	35/35	4/4
E3	Evidence-size sweep	0/25	25/25	25/25	25/25	
E4	Evidence-complexity sweep	0/20	20/20	20/20	20/20	
E5	Validator-count sweep		30/30	24/30		
E6	Tamper detection	4/9	9/9			
E7	Cross-dataset replication	0/36	36/36	36/36	36/36	
E8	Insider tampering	0/7	7/7	7/7	7/7	2/2
E9	Batched Merkle commitments	0/50	50/50	50/50	50/50	4/4

Table 7: Measurements on the live PureChain network (4 validators) before the public RPC became unavailable (E6 runs, 1,000 records each, 100 KB evidence, 50 records/s offered).

Dataset	Committed	rec/s	Incl. P50 (s)	3-conf P50 (s)	tx/block	Reorged	Depth	Verified (%)
momtsim	1,000	36.4	4.30	10.75	66.7	2	1	100.0
momtsim	1,000	41.2	5.14	12.64	90.9	7	3	100.0
momtsim	1,000	27.4	10.37	20.70	76.9	5	3	100.0
saml_d	1,000	34.7	6.86	13.92	83.3	5	3	100.0

Submitted through the load-balanced endpoint; nonce splitting across two RPC nodes inflates latency (see Section 4).

2.2 Evidence integrity (E6)

Across 143,000 applicable malicious modifications, the verification detected 100.00% (95% CI lower bound 99.997%), while 26,000 semantically neutral transformations produced a false-positive rate of 0.00% (upper bound 0.014%). Canonicalisation therefore makes the commitment insensitive to serialisation while any change to content, documents or authorisation invalidates it (Table 8).

2.3 Insider tampering (E8)

Table 10 and Figure 1 show the decisive difference between a trusted database and a blockchain registry. When the insider only alters evidence (S1), every infrastructure detects it because the stored commitment no longer matches. When the insider also rewrites the stored commitment (S2) or inserts a backdated replacement record (S3), the database accepts the writes and the audit passes. On the blockchain configurations the rewrite is rejected by the contract (records are write-once) and a fabricated record carries the block timestamp of its actual inclusion, so both attacks are prevented or detected.

2.4 Batched Merkle commitments (E9)

Committing k records under one Merkle root divides the number of blockchain transactions and the gas per record by k, at the cost of the time a record waits for its batch to close (at most 1 s). Table 11 and Figure 2 report throughput and end-to-end latency as a function of k.

2.5 Cross-dataset replication (E7)

Table 12 repeats the core workloads on the three datasets. The Kruskal–Wallis p-value tests whether the dataset changes the median inclusion latency of an infrastructure; a large p indicates that the results do not depend on the workload source.

2.6 Latency, finality and throughput (E1)

At the common offered load of 50 records/s (W2, 10,000 records), the PoA^2 emulation, Besu and PostgreSQL all sustain the offered load (49.6, 49.7 and 50.0 records/s). Median inclusion latency is 1.56 s for PoA^2, 1.28 s for Besu QBFT and below 5 ms for PostgreSQL. The Ethereum PoS configuration saturates at 16.1 records/s, so its inclusion latency grows to 213.21 s, which is 137 times the PoA^2 value. Finality separates the protocols more sharply (Figure 4): QBFT is final on inclusion (1.28 s), PoA^2 reaches its 6-confirmation finality in 7.55 s, and PoS reaches a finalized checkpoint only after 1,103 s, a factor of 146 slower than PoA^2. Workloads W1–W5 show the same ordering. For PoA^2, throughput and latency are essentially independent of workload volume up to 100,000 records, whereas the Besu latency varies more between workloads and trials (Table 13, Figure 5).

Table 8: Tamper detection against on-chain commitments (E6). Malicious rows report the detection rate (TDR); benign rows report the false-positive rate (FPR). Exact Clopper–Pearson 95% intervals. Pooled over PureChain (live): 4 runs; PoA^2 emulation: 9 runs.

Kind	Modification	Cases	Flagged	Rate (%)	95% CI
benign	reserialize_reorder	13,000	0	0.00	[0.000, 0.028]
benign	unicode_nfd	13,000	0	0.00	[0.000, 0.028]
control	none	13,000	0	0.00	[0.000, 0.028]
malicious	amount	13,000	13,000	100.00	[99.972, 100.000]
malicious	authorization	13,000	13,000	100.00	[99.972, 100.000]
malicious	beneficiary	13,000	13,000	100.00	[99.972, 100.000]
malicious	date	13,000	13,000	100.00	[99.972, 100.000]
malicious	document_deleted	13,000	13,000	100.00	[99.972, 100.000]
malicious	invoice_number	13,000	13,000	100.00	[99.972, 100.000]
malicious	ledger_entry	13,000	13,000	100.00	[99.972, 100.000]
malicious	record_substitution	13,000	13,000	100.00	[99.972, 100.000]
malicious	single_bit_metadata	13,000	13,000	100.00	[99.972, 100.000]
malicious	supporting_document	13,000	13,000	100.00	[99.972, 100.000]
malicious	vendor	13,000	13,000	100.00	[99.972, 100.000]
malicious	ALL (TDR)	143,000	143,000	100.00	[99.997, 100.000]
benign	ALL (FPR)	26,000	0	0.00	[0.000, 0.014]

Table 9: Tamper detection by dataset and infrastructure (E6).

Dataset	Infrastructure	Malicious	TDR (%)	TDR 95% lower	Benign	FPR (%)	FPR 95% upper
IBM AML	PoA^2 emulation	33,000	100.00	99.989	6,000	0.00	0.061
MoMTSim	PoA^2 emulation	33,000	100.00	99.989	6,000	0.00	0.061
MoMTSim	PureChain (live)	33,000	100.00	99.989	6,000	0.00	0.061
SAML-D	PoA^2 emulation	33,000	100.00	99.989	6,000	0.00	0.061
SAML-D	PureChain (live)	11,000	100.00	99.966	2,000	0.00	0.184

2.7 Capacity under increasing load (E2)

Increasing the offered load exposes the capacity of each configuration (Table 14, Figure 6). The PoA^2 emulation sustains 187 records/s with per-record commitments, 3.3 times the 57 records/s of Besu QBFT under the same per-validator CPU budget. At 200 records/s offered, PoA^2 commits 184 records/s with a median inclusion latency of 1.99 s, whereas Besu is saturated at 58 records/s and its latency rises to 46.21 s. PostgreSQL remains the fastest system (1,378 records/s unthrottled), as expected for a single trusted server without replication.

2.8 Validator scaling (E5)

With every validator limited to one CPU, the PoA^2 emulation keeps its throughput at 200 records/s offered as the validator set grows (177, 175 and 175 records/s for 4, 7 and 10 signers). Besu QBFT commits 60 and 54 records/s with 4 and 7 validators; with 10 validators its BFT event queues overflow and the network loses liveness, committing only 4% of the records offered at 50 records/s (Table 15, Figure 7). A fresh confirmation run with 10 validators completed but ran at less than half the offered rate. The all-to-all message pattern of BFT consensus grows quadratically with the validator count, while Clique-style sealing requires one signature per block.

2.9 Evidence size and complexity (E3, E4)

Because only the 32-byte commitment is recorded, commit latency does not depend on the size or complexity of the evidence (Tables 16 and 17). Growth in evidence size and complexity is confined to client-side processing (generation, hashing and off-chain storage), which rises from about 1 ms per record without documents to tens of milliseconds for 5 MB of supporting documents (Figure 8).

2.10 Efficiency: energy, resources and cost

Under the attributed model every four-node blockchain uses 2.03 J per record at 50 records/s, and the PoS configuration 6.25 J (3.1 times more), because its lower throughput keeps the nodes busy longer for the same number of records. The measured dynamic energy of the PoA^2 emulation is 0.050 J per record, 6.0 times lower than Besu QBFT (0.302 J) and below Ethereum PoS (0.171 J); part of this difference comes from the client implementations (Go versus Java). Operational cost without fees is 35.9 µUSD per record for PoA^2, identical to Besu and dominated by the 7-year storage of off-chain evidence. The public-chain configuration adds a gas fee of 2.30 USD per record at the reference price, which PureChain's zero-gas design removes

Table 10: Privileged-insider tampering (E8): share of attacks prevented (write rejected) or detected by the audit, with Clopper–Pearson 95% intervals.

Attack	Infrastructure	Cases	Prevented	Detected	Undetected	Protected (%)
S1 alter evidence	PoA^2 emulation	700	0	700	0	100.0 [99.5, 100.0]
S1 alter evidence	Besu QBFT	700	0	700	0	100.0 [99.5, 100.0]
S1 alter evidence	PostgreSQL	700	0	700	0	100.0 [99.5, 100.0]
S1 alter evidence	Ethereum PoS	200	0	200	0	100.0 [98.2, 100.0]
S2 alter evidence + rewrite commitment	PoA^2 emulation	700	700	0	0	100.0 [99.5, 100.0]
S2 alter evidence + rewrite commitment	Besu QBFT	700	700	0	0	100.0 [99.5, 100.0]
S2 alter evidence + rewrite commitment	PostgreSQL	700	0	0	700	0.0 [0.0, 0.5]
S2 alter evidence + rewrite commitment	Ethereum PoS	200	200	0	0	100.0 [98.2, 100.0]
S3 fabricate + backdate record	PoA^2 emulation	700	0	700	0	100.0 [99.5, 100.0]
S3 fabricate + backdate record	Besu QBFT	700	0	700	0	100.0 [99.5, 100.0]
S3 fabricate + backdate record	PostgreSQL	700	0	0	700	0.0 [0.0, 0.5]
S3 fabricate + backdate record	Ethereum PoS	200	0	200	0	100.0 [98.2, 100.0]

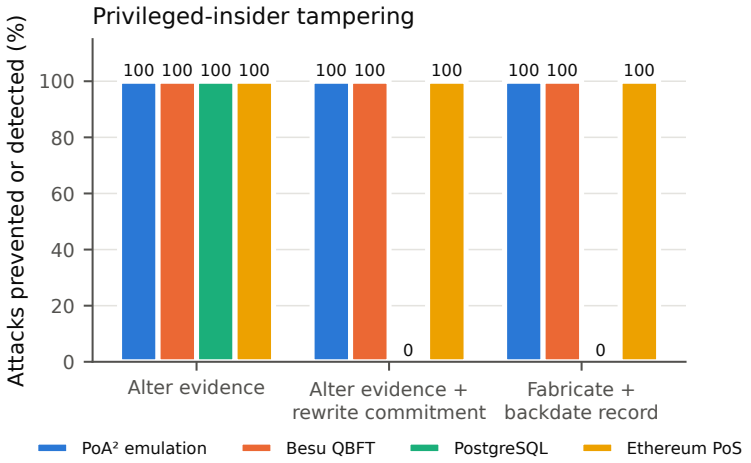


Figure 1: Share of privileged-insider attacks prevented or detected (E8).

entirely (Table 18, Figure 10). Continuous verification of a record takes 14.2 ms on PoA^2, against 33.0 ms on Besu and 5.1 ms on PostgreSQL (Figure 11).

Table 11: Batched Merkle commitments (E9): records per commitment k versus throughput, end-to-end inclusion latency and finality latency (mean over trials, 95% CI).

Infrastructure	Offered	k	n	Throughput (rec/s)	Arrival→incl. P50 (s)	Arrival→final P50 (s)	Gas/record
PoA^2 emulation	50	1	5	48.22 [47.44, 49.00]	1.55 [1.53, 1.58]	7.55 [7.53, 7.56]	153,376
PoA^2 emulation	50	10	5	48.15 [47.00, 49.30]	1.62 [1.60, 1.64]	7.62 [7.60, 7.64]	15,932
PoA^2 emulation	50	25	5	48.13 [47.30, 48.96]	1.77 [1.76, 1.78]	7.77 [7.76, 7.78]	6,373
PoA^2 emulation	50	50	5	47.92 [46.69, 49.15]	2.01 [1.92, 2.10]	8.01 [7.93, 8.10]	3,377
PoA^2 emulation	50	100	5	47.65 [46.64, 48.67]	2.06 [1.97, 2.14]	8.05 [7.97, 8.14]	3,186
PoA^2 emulation	200	1	5	183.24 [178.48, 187.99]	2.10 [1.98, 2.22]	8.10 [7.95, 8.24]	153,377
PoA^2 emulation	200	10	5	189.33 [185.18, 193.49]	1.56 [1.55, 1.57]	7.56 [7.55, 7.57]	15,931
PoA^2 emulation	200	25	5	191.41 [188.24, 194.58]	1.60 [1.58, 1.61]	7.60 [7.58, 7.61]	6,372
PoA^2 emulation	200	50	5	190.47 [186.45, 194.49]	1.66 [1.63, 1.69]	7.66 [7.63, 7.68]	3,186
PoA^2 emulation	200	100	5	190.08 [187.48, 192.67]	1.77 [1.70, 1.83]	7.77 [7.70, 7.84]	1,593
Besu QBFT	50	1	5	43.83 [36.00, 51.67]	7.25 [0.00, 17.43]	7.25 [0.00, 17.43]	153,378
Besu QBFT	50	10	5	48.94 [47.82, 50.07]	0.69 [0.68, 0.71]	0.69 [0.68, 0.71]	15,932
Besu QBFT	50	25	5	48.97 [47.99, 49.95]	0.82 [0.80, 0.85]	0.82 [0.80, 0.85]	6,372
Besu QBFT	50	50	5	48.56 [47.30, 49.82]	1.02 [0.92, 1.12]	1.02 [0.92, 1.12]	3,377
Besu QBFT	50	100	5	48.33 [47.13, 49.53]	1.11 [1.09, 1.13]	1.11 [1.09, 1.13]	3,186
Besu QBFT	200	1	5	51.79 [49.55, 54.04]	50.71 [47.26, 54.16]	50.71 [47.26, 54.16]	153,377
Besu QBFT	200	10	5	194.97 [190.41, 199.54]	0.87 [0.84, 0.89]	0.87 [0.84, 0.89]	15,931
Besu QBFT	200	25	5	195.59 [191.46, 199.73]	0.70 [0.69, 0.71]	0.70 [0.69, 0.71]	6,373
Besu QBFT	200	50	5	195.49 [190.80, 200.18]	0.72 [0.71, 0.74]	0.72 [0.71, 0.74]	3,186
Besu QBFT	200	100	5	196.71 [192.42, 200.99]	0.83 [0.75, 0.91]	0.83 [0.75, 0.91]	1,593
PostgreSQL	50	1	5	49.41 [48.37, 50.44]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	–
PostgreSQL	50	10	5	49.40 [48.37, 50.44]	0.09 [0.09, 0.09]	0.09 [0.09, 0.09]	–
PostgreSQL	50	25	5	49.40 [48.37, 50.44]	0.24 [0.23, 0.25]	0.24 [0.23, 0.25]	–
PostgreSQL	50	50	5	48.95 [47.79, 50.10]	0.49 [0.47, 0.50]	0.49 [0.47, 0.50]	–
PostgreSQL	50	100	5	48.97 [47.78, 50.15]	0.52 [0.51, 0.53]	0.52 [0.51, 0.53]	–
PostgreSQL	200	1	5	199.60 [196.12, 203.07]	0.00 [0.00, 0.01]	0.00 [0.00, 0.01]	–
PostgreSQL	200	10	5	199.59 [196.12, 203.06]	0.03 [0.03, 0.03]	0.03 [0.03, 0.03]	–
PostgreSQL	200	25	5	199.58 [196.11, 203.05]	0.06 [0.06, 0.07]	0.06 [0.06, 0.07]	–
PostgreSQL	200	50	5	199.56 [196.09, 203.03]	0.13 [0.13, 0.13]	0.13 [0.13, 0.13]	–
PostgreSQL	200	100	5	199.54 [196.07, 203.01]	0.26 [0.26, 0.27]	0.26 [0.26, 0.27]	–
Ethereum PoS	50	1	2	15.36 [15.35, 15.38]	67.83 [62.79, 72.88]	1,053 [439.68, 1,666]	153,376
Ethereum PoS	50	50	2	44.92 [44.65, 45.18]	8.72 [8.31, 9.12]	1,105 [1,101, 1,108]	3,375

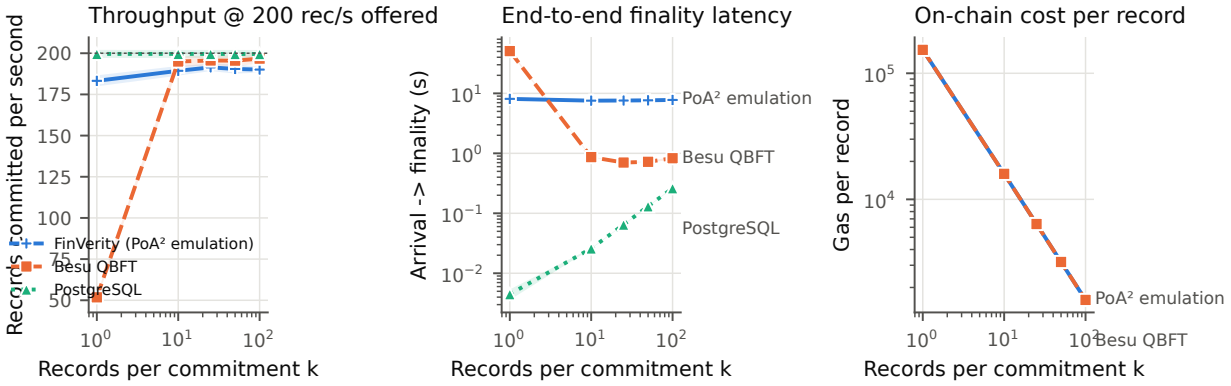


Figure 2: Batched commitments: throughput, arrival-to-finality latency and gas per record (E9).

Table 12: Cross-dataset replication (E1 on MoMTSim, E7 on SAML-D and IBM AML): mean over trials with 95% CI. p: Kruskal–Wallis test of a dataset effect on median inclusion latency.

Workload	Infrastructure	Dataset	n	Throughput (rec/s)	Incl. P50 (s)	Final P50 (s)	Verified (%)	p
W1	Besu QBFT	MoMTSim	30	45.69 [44.25, 47.13]	1.78 [0.97, 2.60]	1.78 [0.97, 2.60]	100.0	0.273
		SAML-D	10	42.98 [38.81, 47.15]	2.96 [0.50, 5.42]	2.96 [0.50, 5.42]	100.0	
		IBM AML	10	45.87 [45.01, 46.73]	1.34 [1.28, 1.40]	1.34 [1.28, 1.40]	100.0	
W1	PostgreSQL	MoMTSim	30	49.58 [49.05, 50.11]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	100.0	0.000
		SAML-D	10	49.28 [48.39, 50.16]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	100.0	
		IBM AML	10	49.28 [48.39, 50.16]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	100.0	
W1	Ethereum PoS	MoMTSim	5	14.71 [14.61, 14.80]	22.54 [21.96, 23.12]	1,126 [1,125, 1,127]	100.0	–
W1	PoA^2 emulation	MoMTSim	30	45.98 [45.48, 46.48]	1.55 [1.54, 1.55]	7.54 [7.53, 7.55]	100.0	0.759
		SAML-D	10	45.87 [45.17, 46.57]	1.55 [1.54, 1.56]	7.53 [7.50, 7.56]	100.0	
		IBM AML	10	45.70 [44.97, 46.43]	1.55 [1.53, 1.56]	7.54 [7.53, 7.55]	100.0	
W2	Besu QBFT	MoMTSim	10	49.69 [49.40, 49.98]	1.28 [1.14, 1.41]	1.28 [1.14, 1.41]	100.0	0.004
		SAML-D	5	49.68 [49.14, 50.21]	1.33 [1.30, 1.37]	1.33 [1.30, 1.37]	100.0	
		IBM AML	5	49.56 [49.00, 50.13]	1.47 [1.35, 1.59]	1.47 [1.35, 1.59]	100.0	
W2	PostgreSQL	MoMTSim	10	50.01 [49.71, 50.30]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	100.0	0.005
		SAML-D	5	50.04 [49.48, 50.60]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	100.0	
		IBM AML	5	50.04 [49.48, 50.60]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	100.0	
W2	Ethereum PoS	MoMTSim	2	16.05 [15.97, 16.13]	213.21 [210.02, 216.40]	1,103 [1,098, 1,107]	100.0	–
W2	PoA^2 emulation	MoMTSim	10	49.63 [49.30, 49.97]	1.56 [1.55, 1.56]	7.55 [7.55, 7.56]	100.0	0.742
		SAML-D	5	49.74 [49.15, 50.32]	1.55 [1.55, 1.56]	7.55 [7.55, 7.56]	100.0	
		IBM AML	5	49.64 [49.12, 50.16]	1.55 [1.55, 1.56]	7.55 [7.54, 7.56]	100.0	
W5	Besu QBFT	MoMTSim	10	49.59 [49.28, 49.90]	2.89 [0.00, 6.37]	2.89 [0.00, 6.37]	100.0	0.089
		SAML-D	3	49.74 [49.45, 50.04]	1.37 [1.30, 1.44]	1.37 [1.30, 1.44]	100.0	
		IBM AML	3	49.66 [49.01, 50.32]	1.59 [1.42, 1.76]	1.59 [1.42, 1.76]	100.0	
W5	PostgreSQL	MoMTSim	10	50.01 [49.71, 50.30]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	100.0	0.349
		SAML-D	3	50.15 [49.74, 50.56]	0.00 [0.00, 0.01]	0.00 [0.00, 0.01]	100.0	
		IBM AML	3	50.15 [49.74, 50.56]	0.00 [0.00, 0.01]	0.00 [0.00, 0.01]	100.0	
W5	PoA^2 emulation	MoMTSim	10	49.64 [49.36, 49.92]	1.55 [1.55, 1.56]	7.55 [7.55, 7.56]	100.0	0.147
		SAML-D	3	49.79 [49.10, 50.49]	1.56 [1.55, 1.56]	7.55 [7.55, 7.56]	100.0	
		IBM AML	3	49.71 [49.28, 50.14]	1.56 [1.55, 1.57]	7.56 [7.55, 7.57]	100.0	

Table 13: Controlled workloads (E1), offered load 50 records/s.

Workload	Infrastructure	n	Success (%)	Throughput (rec/s)	Incl. P50 (s)	Incl. P95 (s)	Final P50 (s)	J/record	µUSD/record
W1	Besu QBFT	30	100.0	45.69 [44.25, 47.13]	1.78 [0.97, 2.60]	2.75 [1.28, 4.22]	1.78 [0.97, 2.60]	2.226	36.01
W1	PostgreSQL	30	100.0	49.58 [49.05, 50.11]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.510	25.69
W1	Ethereum PoS	5	100.0	14.71 [14.61, 14.80]	22.54 [21.96, 23.12]	39.65 [38.99, 40.31]	1,126 [1,125, 1,127]	6.814	2,300,681
W1	PoA^2 emulation	30	100.0	45.98 [45.48, 46.48]	1.55 [1.54, 1.55]	2.00 [2.00, 2.01]	7.54 [7.53, 7.55]	2.188	36.09
W2	Besu QBFT	10	100.0	49.69 [49.40, 49.98]	1.28 [1.14, 1.41]	2.57 [0.71, 4.42]	1.28 [1.14, 1.41]	2.025	35.87
W2	PostgreSQL	10	100.0	50.01 [49.71, 50.30]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.505	25.69
W2	Ethereum PoS	2	100.0	16.05 [15.97, 16.13]	213.21 [210.02, 216.40]	398.73 [389.21, 408.25]	1,103 [1,098, 1,107]	6.249	2,300,688
W2	PoA^2 emulation	10	100.0	49.63 [49.30, 49.97]	1.56 [1.55, 1.56]	2.01 [2.00, 2.01]	7.55 [7.55, 7.56]	2.026	35.93
W3	Besu QBFT	5	100.0	50.00 [49.67, 50.34]	1.34 [1.29, 1.39]	3.85 [0.00, 8.86]	1.34 [1.29, 1.39]	2.013	213.91
W3	PostgreSQL	5	100.0	50.07 [49.73, 50.41]	0.00 [0.00, 0.00]	0.01 [0.01, 0.01]	0.00 [0.00, 0.00]	0.505	203.76
W3	PoA^2 emulation	5	100.0	49.98 [49.64, 50.33]	1.56 [1.56, 1.56]	2.01 [2.01, 2.02]	7.56 [7.55, 7.56]	2.012	213.98
W4	Besu QBFT	3	100.0	50.10 [49.75, 50.45]	1.39 [1.20, 1.58]	2.72 [1.44, 4.00]	1.39 [1.20, 1.58]	2.009	213.90
W4	PostgreSQL	3	100.0	50.14 [49.80, 50.48]	0.01 [0.01, 0.01]	0.01 [0.01, 0.01]	0.01 [0.01, 0.01]	0.505	203.76
W4	PoA^2 emulation	3	100.0	50.10 [49.75, 50.46]	1.56 [1.55, 1.57]	2.01 [2.00, 2.03]	7.56 [7.55, 7.57]	2.007	213.98
W5	Besu QBFT	10	100.0	49.59 [49.28, 49.90]	2.89 [0.00, 6.37]	5.65 [0.00, 13.59]	2.89 [0.00, 6.37]	2.030	2,042
W5	PostgreSQL	10	100.0	50.01 [49.71, 50.30]	0.00 [0.00, 0.00]	0.01 [0.01, 0.01]	0.00 [0.00, 0.00]	0.506	2,032
W5	PoA^2 emulation	10	100.0	49.64 [49.36, 49.92]	1.55 [1.55, 1.56]	2.01 [2.00, 2.01]	7.55 [7.55, 7.56]	2.026	2,042

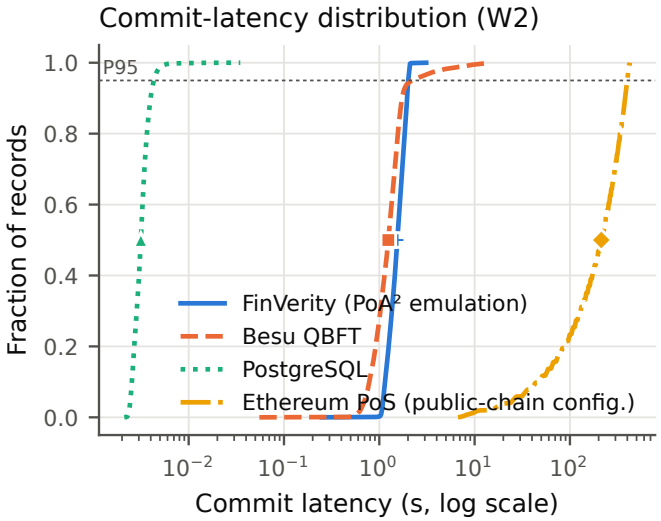


Figure 3: Commit-latency distribution (E1, W2).

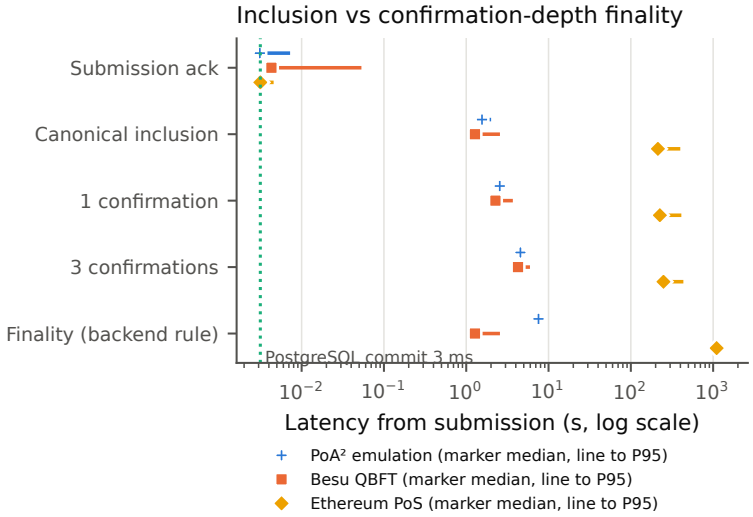


Figure 4: Inclusion versus confirmation-depth finality (E1, W2).

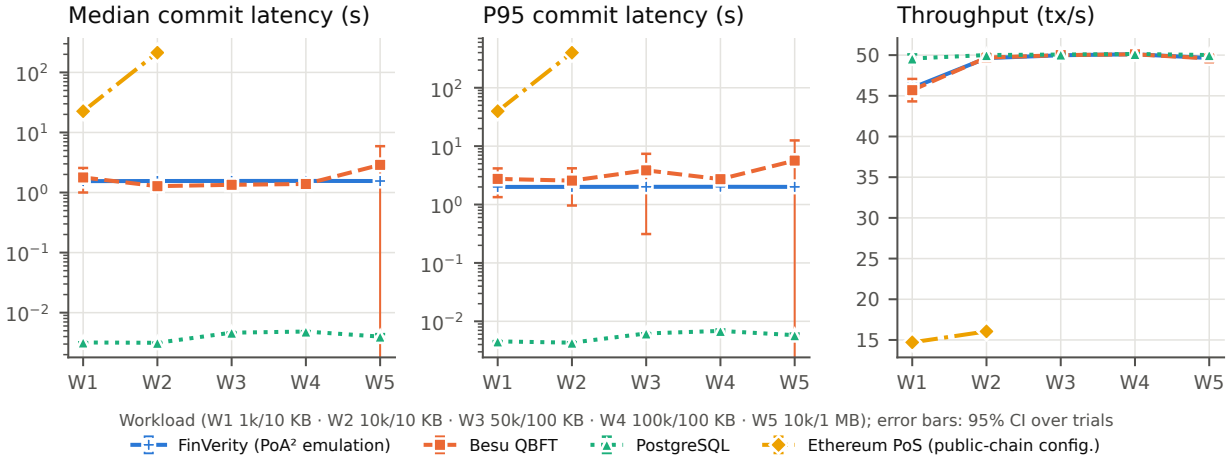


Figure 5: Latency and throughput across workloads W1–W5 (95% CI).

Table 14: Offered-load sweep (E2).

Offered (rec/s)	Infrastructure	n	Success (%)	Throughput (rec/s)	Incl. P50 (s)	Incl. P95 (s)	Final P50 (s)	J/record	µUSD/record
10	Besu QBFT	5	100.0	9.78 [9.54, 10.03]	0.70 [0.67, 0.73]	1.17 [1.11, 1.22]	0.70 [0.67, 0.73]	10.259	37.16
10	PostgreSQL	5	100.0	9.82 [9.57, 10.06]	0.00 [0.00, 0.00]	0.01 [0.01, 0.01]	0.00 [0.00, 0.00]	2.554	25.76
10	Ethereum PoS	2	100.0	8.86 [7.36, 10.36]	7.62 [5.26, 9.98]	13.39 [12.25, 14.52]	1,106 [1,093, 1,118]	11.308	2,300,686
10	PoA² emulation	5	100.0	9.74 [9.51, 9.97]	1.54 [1.53, 1.55]	1.98 [1.97, 1.99]	7.54 [7.53, 7.55]	10.299	37.01
25	Besu QBFT	5	100.0	24.27 [23.67, 24.87]	0.87 [0.84, 0.91]	1.35 [1.31, 1.39]	0.87 [0.84, 0.91]	4.141	36.27
25	PostgreSQL	5	100.0	24.54 [23.94, 25.15]	0.00 [0.00, 0.00]	0.01 [0.00, 0.01]	0.00 [0.00, 0.00]	1.026	25.71
25	PoA² emulation	5	100.0	24.09 [23.43, 24.75]	1.53 [1.52, 1.54]	1.99 [1.98, 1.99]	7.53 [7.52, 7.54]	4.169	36.26
50	Besu QBFT	5	100.0	47.46 [46.29, 48.64]	1.25 [1.21, 1.29]	1.80 [1.72, 1.88]	1.25 [1.21, 1.29]	2.121	35.92
50	PostgreSQL	5	100.0	49.08 [47.87, 50.30]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.515	25.69
50	Ethereum PoS	2	100.0	15.43 [14.41, 16.44]	68.09 [67.19, 69.00]	124.40 [120.91, 127.89]	1,100 [1,080, 1,120]	6.497	2,300,684
50	PoA² emulation	5	100.0	47.51 [46.16, 48.87]	1.56 [1.54, 1.58]	2.00 [1.99, 2.00]	7.55 [7.53, 7.57]	2.118	36.01
100	Besu QBFT	5	100.0	58.12 [55.10, 61.14]	11.34 [8.84, 13.84]	25.78 [22.26, 29.30]	11.34 [8.84, 13.84]	1.736	36.12
100	PostgreSQL	5	100.0	98.81 [96.74, 100.87]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.259	25.68
100	PoA² emulation	5	100.0	94.20 [92.44, 95.95]	1.58 [1.57, 1.59]	2.04 [2.03, 2.05]	7.56 [7.55, 7.57]	1.072	35.84
200	Besu QBFT	5	100.0	57.90 [54.72, 61.08]	46.21 [41.13, 51.29]	73.82 [68.18, 79.47]	46.21 [41.13, 51.29]	1.745	36.52
200	PostgreSQL	5	100.0	199.61 [196.13, 203.08]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.131	25.68
200	PoA² emulation	5	100.0	184.38 [183.91, 184.84]	1.99 [1.79, 2.19]	2.72 [2.34, 3.10]	7.97 [7.76, 8.19]	0.551	35.72
400	Besu QBFT	5	100.0	54.60 [51.64, 57.55]	110.64 [100.27, 121.01]	185.20 [173.79, 196.62]	110.64 [100.27, 121.01]	1.852	36.87
400	PostgreSQL	5	100.0	401.41 [398.17, 404.66]	0.00 [0.00, 0.00]	0.01 [0.00, 0.01]	0.00 [0.00, 0.00]	0.068	25.67
400	PoA² emulation	5	100.0	191.14 [190.50, 191.78]	17.33 [16.94, 17.73]	31.38 [31.05, 31.71]	23.13 [22.76, 23.50]	0.532	35.79
max	Besu QBFT	5	100.0	56.69 [54.85, 58.52]	51.74 [49.13, 54.34]	85.70 [82.25, 89.16]	51.74 [49.13, 54.34]	1.781	36.54
max	PostgreSQL	5	100.0	1,378 [1,344, 1,411]	1.81 [1.76, 1.86]	3.42 [3.34, 3.50]	1.81 [1.76, 1.86]	0.025	25.65
max	PoA² emulation	5	100.0	186.77 [184.21, 189.34]	13.83 [13.42, 14.24]	25.88 [25.53, 26.22]	19.81 [19.44, 20.18]	0.545	35.84

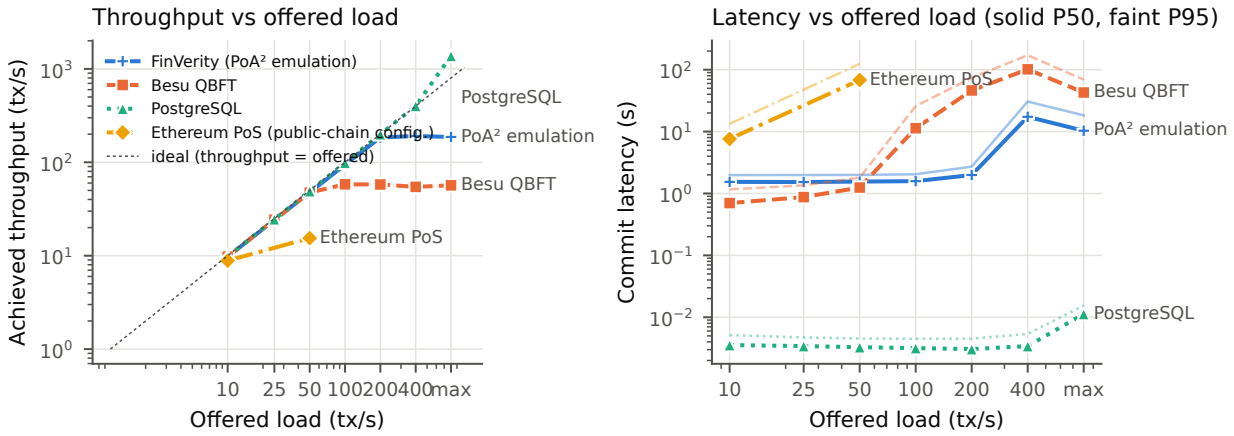


Figure 6: Throughput and latency versus offered load (E2).

Table 15: Validator-count sweep (E5); every validator is limited to 1 CPU. Besu QBFT with 10 validators lost liveness in 3 of 4 trials (BFT event-queue overflow).

Validators @ offered	Infrastructure	n	Success (%)	Throughput (rec/s)	Incl. P50 (s)	Incl. P95 (s)	Final P50 (s)	J/record	µUSD/record
v10 @ 50	Besu QBFT	3	4.0	3.34 [0.00, 17.74]	14.12	31.79	14.12	24.987	536.70
v10 @ 50	PoA² emulation	5	100.0	48.20 [47.25, 49.15]	1.58 [1.57, 1.59]	2.21 [1.95, 2.47]	7.57 [7.55, 7.58]	5.201	52.10
v10 @ 200	PoA² emulation	5	100.0	174.64 [170.51, 178.76]	2.03 [1.79, 2.27]	2.66 [2.34, 2.98]	7.89 [7.70, 8.08]	1.441	51.77
v10 (confirm) @ 50	Besu QBFT	1	100.0	22.53	32.61	46.99	32.61	11.119	53.01
v4 @ 50	Besu QBFT	5	100.0	46.33 [41.08, 51.57]	3.92 [0.00, 11.15]	7.12 [0.00, 21.58]	3.92 [0.00, 11.15]	2.189	35.93
v4 @ 50	PoA² emulation	5	100.0	48.22 [47.22, 49.23]	1.56 [1.55, 1.57]	2.00 [2.00, 2.01]	7.56 [7.55, 7.57]	2.085	35.97
v4 @ 200	Besu QBFT	5	100.0	60.34 [57.82, 62.85]	22.62 [20.72, 24.51]	36.69 [34.22, 39.17]	22.62 [20.72, 24.51]	1.673	36.31
v4 @ 200	PoA² emulation	5	100.0	176.63 [175.08, 178.19]	1.79 [1.68, 1.90]	2.30 [2.17, 2.42]	7.74 [7.64, 7.84]	0.574	35.76
v7 @ 50	Besu QBFT	5	100.0	44.88 [38.06, 51.70]	6.70 [0.00, 18.44]	11.68 [0.00, 32.35]	6.70 [0.00, 18.44]	3.970	44.04
v7 @ 50	PoA² emulation	5	100.0	48.10 [47.18, 49.02]	1.56 [1.52, 1.59]	2.04 [2.03, 2.06]	7.55 [7.52, 7.59]	3.651	44.03
v7 @ 200	Besu QBFT	5	100.0	54.31 [51.92, 56.70]	24.82 [23.89, 25.75]	41.60 [39.40, 43.81]	24.82 [23.89, 25.75]	3.241	44.46
v7 @ 200	PoA² emulation	5	100.0	175.46 [171.04, 179.88]	1.93 [1.75, 2.10]	2.52 [2.33, 2.72]	7.87 [7.71, 8.04]	1.007	43.77

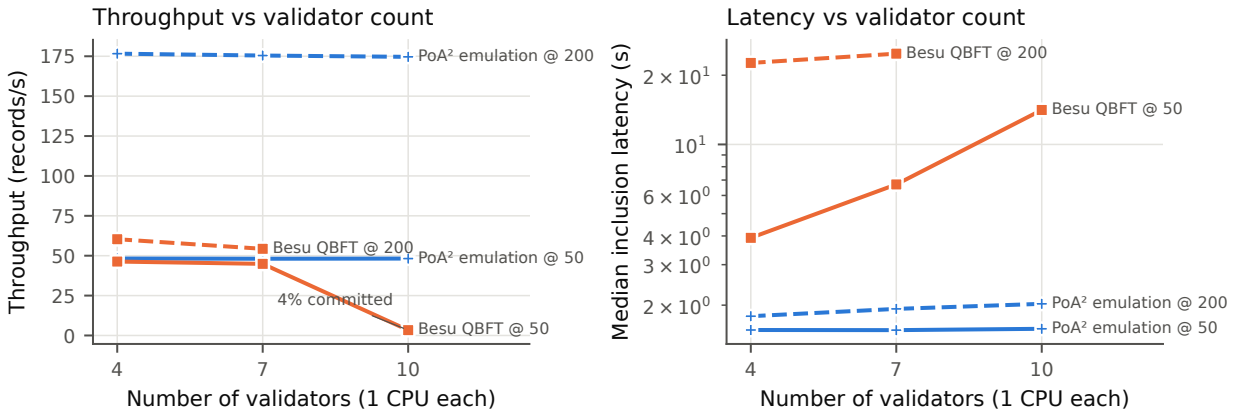


Figure 7: Throughput and latency versus validator count (E5).

Table 16: Evidence-size sweep (E3).

Evidence (bytes)	Infrastructure	n	Success (%)	Throughput (rec/s)	Incl. P50 (s)	Incl. P95 (s)	Final P50 (s)	J/record	µUSD/record
none	Besu QBFT	5	100.0	40.37 [32.11, 48.64]	4.18 [0.00, 9.36]	7.27 [0.00, 16.49]	4.18 [0.00, 9.36]	2.555	15.23
none	PostgreSQL	5	100.0	48.93 [47.71, 50.16]	0.00 [0.00, 0.00]	0.01 [0.00, 0.01]	0.00 [0.00, 0.00]	0.518	4.87
none	PoA² emulation	5	100.0	45.26 [44.24, 46.27]	1.54 [1.53, 1.55]	2.00 [1.99, 2.00]	7.53 [7.52, 7.55]	2.223	15.25
10 KB	Besu QBFT	5	100.0	45.73 [44.13, 47.34]	1.37 [1.27, 1.47]	2.09 [1.84, 2.34]	1.37 [1.27, 1.47]	2.202	36.01
10 KB	PostgreSQL	5	100.0	48.93 [47.71, 50.16]	0.00 [0.00, 0.00]	0.01 [0.00, 0.01]	0.00 [0.00, 0.00]	0.516	25.70
10 KB	PoA² emulation	5	100.0	45.67 [44.30, 47.04]	1.54 [1.50, 1.57]	2.00 [1.99, 2.01]	7.53 [7.50, 7.56]	2.203	36.11
100 KB	Besu QBFT	5	100.0	45.94 [44.68, 47.20]	1.29 [1.21, 1.37]	1.87 [1.82, 1.92]	1.29 [1.21, 1.37]	2.192	214.07
100 KB	PostgreSQL	5	100.0	48.93 [47.71, 50.16]	0.00 [0.00, 0.00]	0.00 [0.00, 0.01]	0.00 [0.00, 0.00]	0.517	203.75
100 KB	PoA² emulation	5	100.0	45.49 [44.21, 46.76]	1.54 [1.52, 1.56]	2.00 [1.99, 2.01]	7.54 [7.52, 7.55]	2.211	214.18
1 MB	Besu QBFT	5	100.0	45.78 [44.09, 47.48]	1.29 [1.22, 1.37]	1.91 [1.72, 2.10]	1.29 [1.22, 1.37]	2.200	2,042
1 MB	PostgreSQL	5	100.0	48.93 [47.71, 50.16]	0.00 [0.00, 0.00]	0.01 [0.01, 0.01]	0.00 [0.00, 0.00]	0.517	2,032
1 MB	PoA² emulation	5	100.0	45.09 [44.10, 46.07]	1.55 [1.53, 1.57]	2.01 [1.99, 2.02]	7.55 [7.53, 7.56]	2.231	2,042
5 MB	Besu QBFT	5	100.0	45.90 [44.65, 47.15]	1.31 [1.27, 1.35]	1.89 [1.76, 2.02]	1.31 [1.27, 1.35]	2.194	10,149
5 MB	PostgreSQL	5	100.0	48.93 [47.71, 50.15]	0.01 [0.01, 0.01]	0.01 [0.01, 0.01]	0.01 [0.01, 0.01]	0.517	10,135
5 MB	PoA² emulation	5	100.0	45.63 [44.71, 46.54]	1.55 [1.53, 1.57]	2.00 [1.99, 2.02]	7.54 [7.52, 7.55]	2.205	10,146

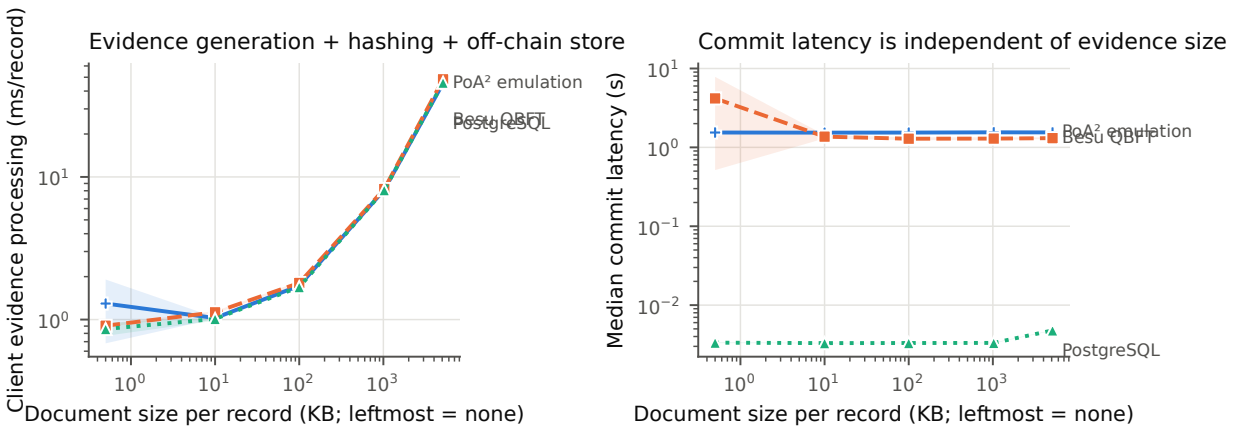


Figure 8: Evidence processing and commit latency versus evidence size (E3).

Table 17: Evidence-complexity sweep (E4).

Level	Infrastructure	n	Success (%)	Throughput (rec/s)	Incl. P50 (s)	Incl. P95 (s)	Final P50 (s)	J/record	µUSD/record
1	Besu QBFT	5	100.0	39.61 [28.77, 50.45]	4.69 [0.00, 10.68]	8.11 [0.00, 19.45]	4.69 [0.00, 10.68]	2.682	13.16
1	PostgreSQL	5	100.0	48.93 [47.71, 50.16]	0.00 [0.00, 0.00]	0.01 [0.00, 0.01]	0.00 [0.00, 0.00]	0.518	2.77
1	PoA^2 emulation	5	100.0	45.13 [43.94, 46.33]	1.56 [1.54, 1.59]	2.01 [2.00, 2.01]	7.56 [7.53, 7.58]	2.228	13.15
2	Besu QBFT	5	100.0	45.82 [44.48, 47.16]	1.36 [1.13, 1.59]	2.17 [1.50, 2.84]	1.36 [1.13, 1.59]	2.198	212.29
2	PostgreSQL	5	100.0	48.93 [47.71, 50.16]	0.00 [0.00, 0.00]	0.01 [0.01, 0.01]	0.00 [0.00, 0.00]	0.517	201.97
2	PoA^2 emulation	5	100.0	45.07 [43.93, 46.20]	1.55 [1.53, 1.58]	2.00 [1.99, 2.01]	7.54 [7.52, 7.57]	2.232	212.39
3	Besu QBFT	5	100.0	46.21 [45.06, 47.36]	1.26 [1.17, 1.34]	1.81 [1.75, 1.87]	1.26 [1.17, 1.34]	2.179	213.17
3	PostgreSQL	5	100.0	48.93 [47.71, 50.15]	0.00 [0.00, 0.00]	0.01 [0.00, 0.01]	0.00 [0.00, 0.00]	0.517	202.86
3	PoA^2 emulation	5	100.0	45.58 [44.50, 46.66]	1.56 [1.53, 1.59]	2.00 [1.99, 2.02]	7.55 [7.53, 7.57]	2.208	213.27
4	Besu QBFT	5	100.0	45.94 [44.23, 47.64]	1.30 [1.26, 1.35]	1.90 [1.81, 1.99]	1.30 [1.26, 1.35]	2.193	214.06
4	PostgreSQL	5	100.0	48.93 [47.71, 50.16]	0.00 [0.00, 0.00]	0.00 [0.00, 0.01]	0.00 [0.00, 0.00]	0.517	203.75
4	PoA^2 emulation	5	100.0	45.58 [44.16, 46.99]	1.54 [1.51, 1.57]	2.00 [1.99, 2.02]	7.53 [7.50, 7.56]	2.207	214.16

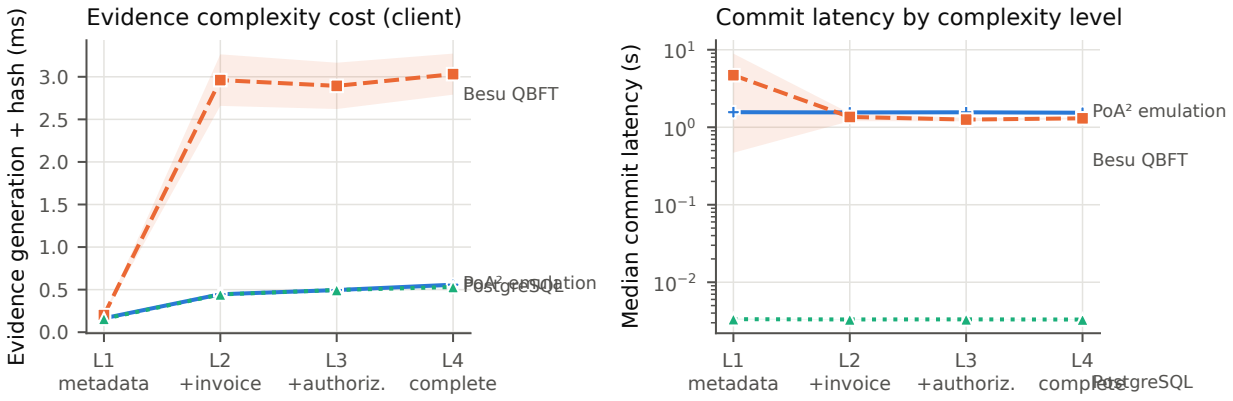


Figure 9: Effect of evidence complexity (E4). The client code is identical for every backend; the higher client cost during Besu trials reflects CPU contention from the co-located Java validators.

Table 18: Efficiency metrics (MoMTSim workloads): CPU in cores used on average during the processing window, peak infrastructure memory, network I/O, storage overhead per record, energy and operational cost. n/m: not measurable (remote validators).

Workload	Infrastructure	CPU client	CPU infra	Mem (MB)	Net (KB/rec)	On-chain (B/rec)	Off-chain (KB/rec)	E attr. (J)	E dyn. (J)	µUSD/rec
W1	Besu QBFT	0.12	2.48	3,140	9.97	397	12.3	2.226	0.3125	36.01
W1	PostgreSQL	0.05	0.02	80	0.90	182	12.3	0.510	0.0076	25.69
W1	Ethereum PoS	0.04	0.30	835	47.99	402	12.3	6.814	0.1261	2,300,681
W1	PoA^2 emulation	0.10	0.30	1,207	10.43	398	12.3	2.188	0.0466	36.09
W2	Besu QBFT	0.12	2.65	5,209	8.42	397	12.3	2.025	0.3016	35.87
W2	PostgreSQL	0.05	0.02	87	0.94	182	12.3	0.505	0.0077	25.69
W2	Ethereum PoS	0.06	0.45	1,012	56.65	402	12.3	6.249	0.1708	2,300,688
W2	PoA^2 emulation	0.10	0.36	2,726	8.69	398	12.3	2.026	0.0500	35.93
W5	Besu QBFT	0.12	2.76	5,217	8.45	397	1,026	2.030	0.3151	2,042
W5	PostgreSQL	0.05	0.02	105	0.94	182	1,026	0.506	0.0081	2,032
W5	PoA^2 emulation	0.10	0.36	2,597	8.68	398	1,026	2.026	0.0508	2,042

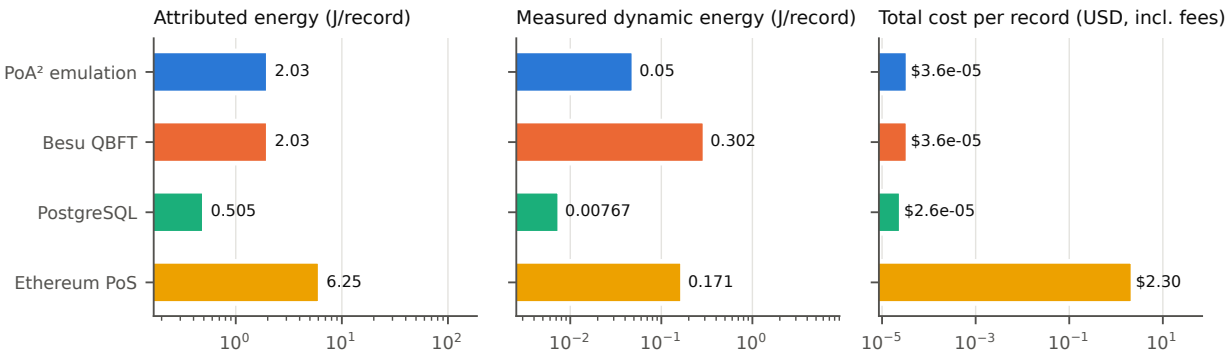


Figure 10: Energy and total cost per record (E1, W2), including gas fees.

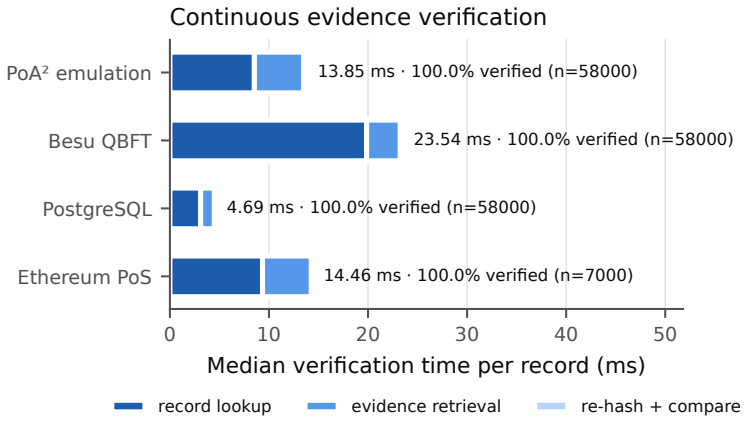


Figure 11: Continuous verification time per record.

3 Summary of findings

The research question asked whether a lightweight, zero-gas, PoA^2-based permissioned blockchain can provide cryptographically verifiable audit evidence while reducing latency, energy consumption and operational cost.

- **Verifiable evidence.** Every committed record verified; 100.00% of 143,000 malicious modifications were detected with no false positives, on all three datasets. Against a privileged insider, the blockchain registry prevented or detected 100% of commitment rewrites and backdated records, while the centralised database detected none of them.
- **Versus the conventional public-chain configuration.** PoA^2 lowers inclusion latency by a factor of 137, finality latency by a factor of 146 (7.55 s versus 1,103 s), attributed energy by a factor of 3.1, and removes the gas fee of 2.30 USD per record.
- **Versus the conventional permissioned blockchain.** PoA^2 sustains 3.3 times the per-record capacity of Besu QBFT, keeps its throughput from 4 to 10 validators where QBFT loses liveness, and uses 6.0 times less measured dynamic energy. QBFT provides immediate finality, whereas PoA^2 needs 6 confirmations (7.55 s).
- **Versus the centralised database.** The database is faster and about 30% cheaper per record, but it provides no protection against an insider who controls it; the blockchain overhead buys independent verifiability.
- **Robustness.** The results do not depend on the workload dataset (smallest Kruskal–Wallis p for PoA^2: 0.15) or on evidence size and complexity. Batched Merkle commitments cut gas per record from 153,376 to 2,390 and let even the PoS configuration raise throughput from 15.4 to 44.9 records/s, but they do not change its finality latency.

4 Threats to validity

- **Availability of the live network.** The public PureChain RPC endpoint stopped producing blocks during the campaign (last block 1,807,036) while all four validators were still sealing immediately before the halt. The controlled experiments were therefore completed on a calibrated PoA^2 emulation; live measurements are reported separately and will be extended when the endpoint recovers.
- **Emulation fidelity.** The emulation reproduces validator count, block period, gas limit, fee-free operation and Clique-style sealing, but not PureChain's modified fork rules, the contextual-association component of PoA^2, the fork rate observed on the live network under load, or the wide-area network path.
- **Live network stability.** Under sustained load of about 50 transactions/s the live network produced competing blocks (23.5% of observed blocks) and then halted; this is a property of the deployment at the time of measurement and is reported, not excluded.
- **Load-balanced endpoint.** The public endpoint alternates requests between two RPC nodes, so consecutive nonces of one account can reach different nodes and be delayed. Live latencies are therefore conservative.
- **Single host.** Local networks share one workstation; per-validator CPU quotas and backend isolation limit, but do not remove, co-location effects.
- **Energy model.** Remote validator energy cannot be metered; the attributed model uses a fixed node power and is reported with the measured dynamic energy of the client and local containers.
- **Synthetic evidence.** Audit evidence is generated synthetically because real audit evidence is confidential; document contents do not affect hashing cost, which depends only on size.

Data and code availability

Workload preprocessing, evidence generation, contract, load generator, analysis and this report are produced by the FinVerity simulation framework; every table and figure is regenerated from the raw trial data with `python -m analysis.report`.

References

[1] Erik Altman et al. Realistic synthetic financial transactions for anti-money laundering models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[2] Denish Azamuke. Synthetic mobile money transaction dataset (MoMTSim), version 2. Mendeley Data, doi:10.17632/zhj366m53p.2, 2024. Related article: IEEE Access, doi:10.1109/ACCESS.2024.3439012.

[3] Vitalik Buterin et al. EIP-1559: Fee market change for ETH 1.0 chain. Ethereum Improvement Proposals, 2019.

[4] Xiaobo Fan, Wolf-Dietrich Weber, and Luiz André Barroso. Power provisioning for a warehouse-sized computer. In *Proc. 34th International Symposium on Computer Architecture (ISCA)*, 2007.

[5] Hyperledger Foundation. Hyperledger besu: QBFT consensus. Besu documentation, 2024.

[6] Berkan Oztas et al. SAML-D: Synthetic transaction monitoring dataset for anti-money laundering. Kaggle, berkanoztas/synthetic-transaction-monitoring-dataset-aml, 2023.

[7] A. Rundgren, B. Jordan, and S. Erdtman. JSON canonicalization scheme (JCS). RFC 8785, IETF, 2020.

[8] Johannes Sedlmeir, Hans Ulrich Buhl, Gilbert Fridgen, and Robert Keller. The energy consumption of blockchain technology: Beyond myth. *Business & Information Systems Engineering*, 62:599–608, 2020.

[9] Martin Holst Swende and Nick Johnson. EIP-191: Signed data standard. Ethereum Improvement Proposals, 2016.

[10] Péter Szilágyi. EIP-225: Clique proof-of-authority consensus protocol. Ethereum Improvement Proposals, 2017.